Lab Program 01: Genetic Algorithm

```
import random


EXAMS = ["Math", "Science", "History", "English"]

TIME_SLOTS = [0, 1, 2]  # Slot 0, Slot 1, Slot 2

STUDENTS = [

    {"name": "Student A", "courses": ["Math", "Science"]},

    {"name": "Student B", "courses": ["Science", "History"]}

]


def evaluate_fitness(schedule):

    """

    Grades a schedule. Perfect schedule = 100.

    Each student conflict subtracts 25 points.

    """

    clashes = 0

    timetable = {EXAMS[i]: schedule[i] for i in range(len(EXAMS))}


    for student in STUDENTS:

                    slots_used  =  [timetable[course]  for  course  in
student["courses"]]


        if len(slots_used) != len(set(slots_used)):

            clashes += 1


    return 100 - (clashes * 25)
```

```
total_schedules_to_track = 10

chance_to_mix_schedules  = 0.8

chance_to_tweak_exam_slot = 0.2

total_cycles_to_run      = 20



best_schedule = None

best_fitness = -1

population = []

for _ in range(total_schedules_to_track):



          random_schedule   =   [random.choice(TIME_SLOTS)   for   _   in
range(len(EXAMS))]

    population.append(random_schedule)



print("--- Starting Evolution Loop ---\n")

for cycle in range(1, total_cycles_to_run + 1):



    fitness_scores = [evaluate_fitness(sched) for sched in population]



    for i in range(total_schedules_to_track):

        if fitness_scores[i] > best_fitness:

            best_fitness = fitness_scores[i]

            best_schedule = population[i].copy()



            print(f"Cycle   {cycle}:   Best   Fitness   Score   Found   =
{best_fitness}/100")



    if best_fitness == 100:
```

```
        print("-> Success! A perfect schedule was found early.")
        break

    next_generation = []
    while len(next_generation) < total_schedules_to_track:

            r1, r2 = random.randint(0, total_schedules_to_track-1), random.randint(0, total_schedules_to_track-1)
                parent1 = population[r1] if fitness_scores[r1] >= fitness_scores[r2] else population[r2]

            r3, r4 = random.randint(0, total_schedules_to_track-1), random.randint(0, total_schedules_to_track-1)
                parent2 = population[r3] if fitness_scores[r3] >= fitness_scores[r4] else population[r4]

        if random.random() < chance_to_mix_schedules:
            split_point = random.randint(1, len(EXAMS) - 1)
            child1 = parent1[:split_point] + parent2[split_point:]
            child2 = parent2[:split_point] + parent1[split_point:]
        else:
            child1 = parent1.copy()
            child2 = parent2.copy()
        if random.random() < chance_to_tweak_exam_slot:
            mutate_idx = random.randint(0, len(EXAMS) - 1)
            child1[mutate_idx] = random.choice(TIME_SLOTS)

        if random.random() < chance_to_tweak_exam_slot:
```

```
            mutate_idx = random.randint(0, len(EXAMS) - 1)
            child2[mutate_idx] = random.choice(TIME_SLOTS)

        next_generation.append(child1)
        next_generation.append(child2)

    population = next_generation[:total_schedules_to_track]

print("\n--- Final Optimization Result ---")
print(f"Final Best Fitness Score: {best_fitness}/100")
print("Optimized Timetable Schedule:")
for i, exam in enumerate(EXAMS):
            print(f"  * {exam} Exam -> Assigned to Time Slot {best_schedule[i]}")
```

Output:

```
•••  --- Starting Evolution Loop ---

     Cycle 1: Best Fitness Score Found = 100/100
     -> Success! A perfect schedule was found early.

     --- Final Optimization Result ---
     Final Best Fitness Score: 100/100
     Optimized Timetable Schedule:
      * Math Exam -> Assigned to Time Slot 1
      * Science Exam -> Assigned to Time Slot 2
      * History Exam -> Assigned to Time Slot 1
      * English Exam -> Assigned to Time Slot 0
```